

Tugas 5

Struktur Data

Red Black Tree



Dosen Pengampu :

M. Chandra Cahyo Utomo, S. Kom., M. Kom.

NIP. 199202202019031000

Disusun Oleh Kelompok 2 :

Cindy Aulia Renita	11241023
Fauzan Fayzul Haq	11241028
Thisya Darmala	11241083
Zharif Aziz Zulkarnain Widodo	11231092

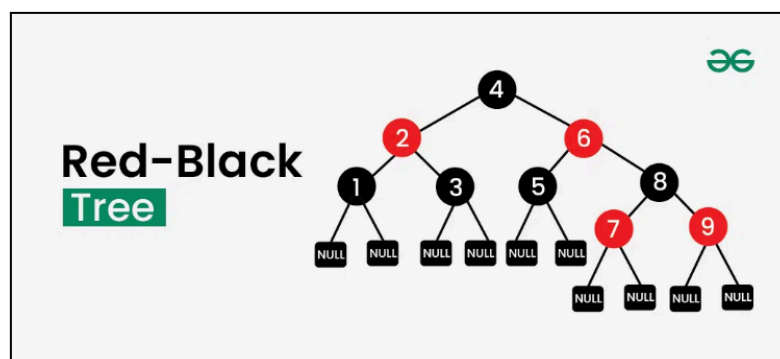
21 November 2025

Dasar Teori

Binary search trees adalah struktur data dasar, tetapi kinerjanya bisa menurun jika pohon menjadi tidak seimbang atau tinggi pohon bisa menjadi $O(n)$. *Red-Black Tree* adalah *self-balancing binary search tree* di mana tinggi pohon tidak pernah melebihi $O(\log n)$.

Sr. No.	Algorithm	Time Complexity
1.	Search	$O(\log n)$
2.	Insert	$O(\log n)$
3.	Delete	$O(\log n)$

Setiap node memiliki atribut tambahan: sebuah warna, yang bisa *red* atau *black*. Warna-warna ini digunakan untuk menjaga keseimbangan saat operasi *insert* dan *delete*, sehingga memastikan pengambilan dan manipulasi data tetap efisien.

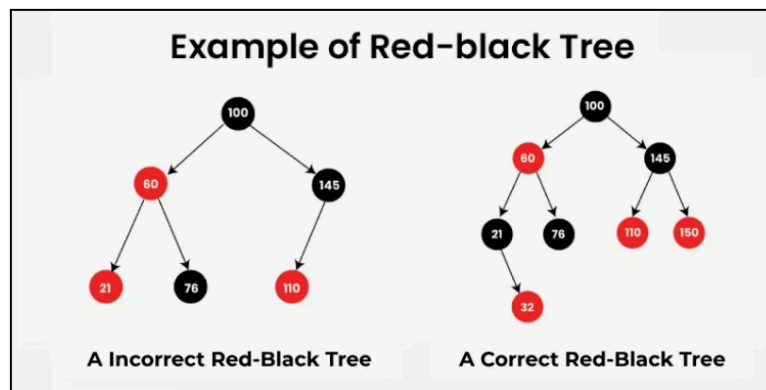


Properti pada *Red-Black Tree* :

- 1) *Node Color*: Setiap node berwarna *red* atau *black*.
- 2) *Root Property*: Root dari pohon selalu berwarna *black*.
- 3) *Red Property*: Node *red* tidak boleh memiliki anak berwarna *red* (tidak boleh ada dua node merah berturut-turut pada jalur mana pun).
- 4) *Black Property*: Setiap jalur dari sebuah node ke *descendant* null nodes (*leaves*) memiliki jumlah node *black* yang sama.
- 5) *Leaf Property*: Semua *leaves* (NIL nodes) berwarna *black*.

Properti-properti ini memastikan bahwa jalur terpanjang dari root ke sebuah *leaf* tidak lebih dari dua kali panjang jalur terpendek, sehingga menjaga keseimbangan pohon dan kinerja yang efisien.

Contoh *Red-Black Tree* :



Correct Red-Black Tree pada gambar memastikan bahwa setiap jalur dari root ke sebuah *leaf* node memiliki jumlah node black yang sama. Dalam contoh ini ada satu (tidak termasuk root). *Incorrect Red Black Tree* tidak memenuhi properti *red-black* karena dua node *red* berdampingan. Masalah lain adalah salah satu jalur ke *leaf* node memiliki nol node *black*, sedangkan dua jalur lainnya mengandung sebuah node *black*.

Source Code

No	Node.java
1	<code>package main;</code>
2	
3	<code>public class Node {</code>
4	<code> private int key;</code>
5	<code> private Node right;</code>
6	<code> private Node left;</code>
7	<code> private Node parent;</code>
8	<code> private boolean red;</code>
9	
10	<code> public Node(int key) {</code>
11	<code> this.key = key;</code>
12	<code> }</code>
13	<code> public int getKey() {</code>
14	<code> return key;</code>
15	<code> }</code>
16	
17	<code> public void setKey(int key) {</code>
18	<code> this.key = key;</code>
19	<code> }</code>
20	
21	<code> public Node getRight() {</code>
22	<code> return right;</code>
23	<code> }</code>
24	
25	<code> public void setRight(Node right) {</code>
26	<code> this.right = right;</code>
27	<code> }</code>
28	
29	<code> public Node getLeft() {</code>
30	<code> return left;</code>
31	<code> }</code>
32	<code> public void setLeft(Node left) {</code>
33	<code> this.left = left;</code>
34	<code> }</code>
35	
36	<code> public Node getParent() {</code>
37	<code> return parent;</code>
38	<code> }</code>
39	
40	<code> public void setParent(Node parent) {</code>
41	<code> this.parent = parent;</code>
42	<code> }</code>
41	
42	<code> public boolean isRed() {</code>

43	return red;
44	}
45	
46	public void setRed(boolean red) {
47	this.red = red;
48	}
49	}

No	Orders.java
1	package main;
2	
3	public class Orders {
4	private void preOrderHelper(Node node) {
5	if (node != null) {
6	System.out.print(node.getKey() + " ");
7	preOrderHelper(node.getLeft());
8	preOrderHelper(node.getRight());
9	}
10	}
11	public void preorder(Node root) {
12	preOrderHelper(root);
13	}
14	
15	private void inOrderHelper(Node node) {
16	if (node != null) {
17	inOrderHelper(node.getLeft());
18	System.out.print(node.getKey() + " ");
19	inOrderHelper(node.getRight());
20	}
21	}
22	
23	public void inorder(Node root) {
24	inOrderHelper(root);
25	}
26	
27	private void postOrderHelper(Node node) {
28	if (node != null) {
29	postOrderHelper(node.getLeft());
30	postOrderHelper(node.getRight());
31	System.out.print(node.getKey() + " ");
32	}
33	}
34	public void postorder(Node root) {
35	postOrderHelper(root);
36	}

No	RedBlackTree.java
1	<code>package main;</code>
2	
3	<code>public class RedBlackTree {</code>
4	<code> private Node root;</code>
5	
6	<code> public RedBlackTree() {</code>
7	<code> this.root = null;</code>
8	<code> }</code>
9	
10	<code> public Node getRoot() {</code>
11	<code> return this.root;</code>
12	<code> }</code>
13	<code> private void rotateLeft(Node node) {</code>
14	<code> Node rightChild = node.getRight();</code>
15	<code> node.setRight(rightChild.getLeft());</code>
16	
17	<code> if (rightChild.getLeft() != null) {</code>
18	<code> rightChild.getLeft().setParent(node);</code>
19	<code> }</code>
20	
21	<code> rightChild.setParent(node.getParent());</code>
22	
23	<code> if (node.getParent() == null) {</code>
24	<code> this.root = rightChild;</code>
25	<code> } else if (node == node.getParent().getLeft()) {</code>
26	<code> node.getParent().setLeft(rightChild);</code>
27	<code> } else {</code>
28	<code> node.getParent().setRight(rightChild);</code>
29	<code> }</code>
30	<code> rightChild.setLeft(node);</code>
31	<code> node.setParent(rightChild);</code>
32	<code> }</code>
33	
34	<code> private void rotateRight(Node node) {</code>
35	<code> Node leftChild = node.getLeft();</code>
36	<code> node.setLeft(leftChild.getRight());</code>
37	
38	<code> if (leftChild.getRight() != null) {</code>
39	<code> leftChild.getRight().setParent(node);</code>
40	<code> }</code>
41	
42	<code> leftChild.setParent(node.getParent());</code>

```

41         if (node.getParent() == null) {
42             this.root = leftChild;
43         } else if (node == node.getParent().getRight()) {
44             node.getParent().setRight(leftChild);
45         } else {
46             node.getParent().setLeft(leftChild);
47         }
48
49         leftChild.setRight(node);
50         node.setParent(leftChild);
51     }
52
53     public void add(int key) {
54         Node newNode = new Node(key);
55         newNode.setRed(true);
56
57         if (this.root == null) {
58             this.root = newNode;
59             newNode.setRed(false);
60             return;
61         }
62
63         Node parent = null;
64         Node current = this.root;
65
66         while (current != null) {
67             parent = current;
68             // Changed compareTo to standard integer
69             operators if (newNode.getKey() < current.getKey()) {
70                 current = current.getLeft();
71             } else {
72                 current = current.getRight();
73             }
74         }
75
76         newNode.setParent(parent);
77         // Changed compareTo to standard integer operators
78         if (newNode.getKey() < parent.getKey()) {
79             parent.setLeft(newNode);
80         } else {
81             parent.setRight(newNode);
82         }
83
84         fixInsert(newNode);
85     }
86
87     private void fixInsert(Node node) {
88         while (node.getParent() != null &&
89             node.getParent().isRed()) {

```

```

88         if (node.getParent() ==
89 node.getParent().getParent().getLeft()) {
90             Node uncle =
91 node.getParent().getParent().getRight();
92
93             if (uncle != null && uncle.isRed()) {
94                 node.getParent().setRed(false);
95                 uncle.setRed(false);
96
97 node.getParent().getParent().setRed(true);
98                 node = node.getParent().getParent();
99             } else {
100                 if (node == node.getParent().getRight())
101 {
102                     node = node.getParent();
103                     rotateLeft(node);
104                 }
105                 node.getParent().setRed(false);
106 node.getParent().getParent().setRed(true);
107
108 rotateRight(node.getParent().getParent());
109             }
110             } else {
111                 Node uncle =
112 node.getParent().getParent().getLeft();
113
114                 if (uncle != null && uncle.isRed()) {
115                     node.getParent().setRed(false);
116                     uncle.setRed(false);
117
118 node.getParent().getParent().setRed(true);
119                     node = node.getParent().getParent();
120                 } else {
121                     if (node == node.getParent().getLeft())
122 {
123                     node = node.getParent();
124                     rotateRight(node);
125                 }
126                 node.getParent().setRed(false);
127 node.getParent().getParent().setRed(true);
128
129 rotateLeft(node.getParent().getParent());
130             }
131         }
132
133         if (node == this.root) {
134             break;
135         }

```


135	}
136	this.root.setRed(false);
137	}
138	}

No	TreeVisualizer.java
1	package main;
2	
3	import java.util.*;
4	
5	public class TreeVisualizer {
6	
7	public static final String ANSI_RESET = "\u001B[0m";
8	public static final String ANSI_RED = "\u001B[31m";
9	public static final String ANSI_BLACK_OR_GREY =
10	"\u001B[70m";
11	
12	public void printCompact(Node root) {
13	if (root == null) {
14	System.out.println("Empty tree");
15	return;
16	}
17	
18	System.out.println("\n=== Red-Black Tree ===");
19	List<List<String>> lines = new ArrayList<>();
20	List<Node> level = new ArrayList<>();
21	List<Node> next = new ArrayList<>();
22	
23	level.add(root);
24	int nn = 1;
25	int widest = 0;
26	
27	while (nn != 0) {
28	List<String> line = new ArrayList<>();
29	nn = 0;
30	
31	for (Node n : level) {
32	if (n == null) {
33	line.add(null);
34	next.add(null);
35	next.add(null);
36	} else {
37	String s = String.valueOf(n.getKey());
38	// Apply color based on node's redness
39	if (n.isRed()) {
	s = ANSI_RED + s + ANSI_RESET;

```

40         } else {
41             s = ANSI_BLACK_OR_GREY + s +
42 ANSI_RESET;
41         }
42
43         // We need to calculate widest based on
44 the actual key length, not the colored string length
45         if (String.valueOf(n.getKey()).length()
46 > widest) widest = String.valueOf(n.getKey()).length();
47
48         line.add(s);
49
50         next.add(n.getLeft());
51         next.add(n.getRight());
52
53         if (n.getLeft() != null) nn++;
54         if (n.getRight() != null) nn++;
55     }
56
57     if (widest % 2 == 1) widest++;
58
59     lines.add(line);
60
61     List<Node> tmp = level;
62     level = next;
63     next = tmp;
64     next.clear();
65 }
66
67     int perpiece = lines.get(lines.size() - 1).size() *
68 (widest + 4);
69     for (int i = 0; i < lines.size(); i++) {
70         List<String> line = lines.get(i);
71         int hpw = (int) Math.floor(perpiece / 2f) - 1;
72
73         if (i > 0) {
74             for (int j = 0; j < line.size(); j++) {
75                 char c = ' ';
76                 if (j % 2 == 1) {
77                     if (line.get(j - 1) != null &&
78 !line.get(j - 1).trim().isEmpty()) { // Check for actual
79 content
80                         c = (line.get(j) != null &&
81 !line.get(j).trim().isEmpty()) ? '└' : '┘'; // Check for
82 actual content
83                     } else {
84                         if (line.get(j) != null &&
85 !line.get(j).trim().isEmpty()) c = '┌'; // Check for actual
86 content

```

```

85         }
86     }
87     System.out.print(c);
88
89     if (line.get(j) == null ||
90 line.get(j).trim().isEmpty()) { // Check for actual content
91         for (int k = 0; k < perpiece - 1;
92 k++) {
93             System.out.print(" ");
94         }
95     } else {
96         for (int k = 0; k < hpw; k++) {
97             System.out.print(j % 2 == 0 ? "
98 " : "-");
99         }
100        System.out.print(j % 2 == 0 ? "┌" :
101 "└");
102        for (int k = 0; k < hpw; k++) {
103            System.out.print(j % 2 == 0 ?
104 "- " : " ");
105        }
106    }
107    System.out.println();
108 }
109
110 for (int j = 0; j < line.size(); j++) {
111     String f = line.get(j);
112     if (f == null) f = "";
113
114     int actualKeyLength = 0;
115     if (line.get(j) != null &&
116 !line.get(j).trim().isEmpty()) {
117         String rawKey =
118 line.get(j).replaceAll("\\u001B\\[[;\\d]*m", ""); // Remove
119 ANSI codes
120         actualKeyLength = rawKey.length();
121     }
122
123     int gap1 = (int) Math.ceil(perpiece / 2f -
124 actualKeyLength / 2f);
125     int gap2 = (int) Math.floor(perpiece / 2f -
126 actualKeyLength / 2f);
127
128     for (int k = 0; k < gap1; k++) {
129         System.out.print(" ");
130     }
131     System.out.print(f);
132     for (int k = 0; k < gap2; k++) {
133         System.out.print(" ");

```

132	}
133	}
134	System.out.println();
135	
136	perpiece /= 2;
137	}
138	System.out.println();
139	}
140	}

No	Main.java
1	package main;
2	
3	import java.util.Scanner;
4	import java.util.Random;
5	
6	public class Main {
7	public static void main(String[] args) {
8	Scanner scanner = new Scanner(System.in);
9	Random random = new Random();
10	RedBlackTree rdt = new RedBlackTree();
10	Orders orders = new Orders();
11	TreeVisualizer viz = new TreeVisualizer();
12	
13	boolean running = true;
14	showMenu();
15	while (running) {
16	System.out.print("==== Choose: ");
17	int choice;
18	try{
19	choice = scanner.nextInt();
20	} catch (Exception e){
21	scanner.nextLine();
22	choice = -1;
23	}
24	switch (choice) {
25	case 1:
26	System.out.print("Enter number: ");
27	int num = scanner.nextInt();
28	rdt.add(num);
29	System.out.println(num + " added.");
30	break;
31	case 2:
32	rdt = new RedBlackTree();
33	System.out.print("How many random

```

34 numbers to generate? ");
35         int count = scanner.nextInt();
36         System.out.print("Adding: ");
37         for (int i = 0; i < count; i++) {
38             int val = random.nextInt(100) + 1;
39             System.out.print(val + " ");
40             rdt.add(val);
41         }
42         System.out.println("\nTree
43 regenerated.");
44         break;
45
46     case 3:
47         if (rdt.getRoot() == null) {
48             System.out.println("Tree is
49 empty.");
50         } else {
51             System.out.println("\n--- Visual
52 Representation ---");
53             viz.printCompact(rdt.getRoot());
54             System.out.println("\nPreorder
55 Traversal:");
56             orders.preorder(rdt.getRoot());
57             System.out.println("\nInOrder
58 Traversal:");
59             orders.inorder(rdt.getRoot());
60             System.out.println("\nPostorder
61 Traversal:");
62             orders.postorder(rdt.getRoot());
63             System.out.println();
64         }
65         break;
66
67     case 4:
68         running = false;
69         break;
70
71     default:
72         System.out.println("Invalid option.
73 Please retry");
74         showMenu();
75     }
76 scanner.close();
77 }
78
79 private static void showMenu() {

```

```
79         System.out.println("\n=== Red Black Tree Menu ===");
80         System.out.println("1. Tambah data");
81         System.out.println("2. Reset dan generate random");
82         System.out.println("3. Show Tree & Traversals");
83         System.out.println("4. Exit");
84     }
85 }
```

Screenshot

1. Tampilan Menu

Program menampilkan menu dengan empat pilihan: Tambah data, *Reset* dan *generate random*, *Show Tree & Traversals*, dan *Exit*. Pengguna memilih opsi dengan memasukkan angka menu.

```
=== Red Black Tree Menu ===
1. Tambah data
2. Reset dan generate random
3. Show Tree & Traversals
4. Exit
==== Choose:
```

2. Insert Angka (Opsi 1)

Opsi ini mengosongkan pohon (mengatur root ke NIL) lalu membuat serangkaian angka acak yang dimasukkan satu per satu. Setiap angka diproses dengan mekanisme insert diikuti langkah perbaikan yang sama, sehingga pohon hasil *generate* tetap memenuhi aturan *Red-Black* dan dapat dipakai untuk menguji stabilitas struktur pada berbagai data.

```
=== Red Black Tree Menu ===
1. Tambah data
2. Reset dan generate random
3. Show Tree & Traversals
4. Exit
==== Choose: 1
Enter number: 12
12 added.
==== Choose: 1
Enter number: 56
56 added.
==== Choose: 1
Enter number: 61
61 added.
==== Choose:
```

3. Visualisasi dan *Traversals* Angka *Insert* (Opsi 3)

Program menampilkan representasi teks pohon dengan indentasi untuk menunjukkan relasi *parent-child* dan mencetak tiga traversal standar: *Preorder*, *InOrder*, dan *Postorder*. *Traversal* dibuat secara rekursif tanpa mengubah struktur; khususnya *InOrder* menampilkan elemen dalam urutan menaik, berguna untuk memeriksa bahwa sifat *BST* (*Binary Search Tree*) tetap terjaga.

```
==== Choose: 3

--- Visual Representation ---

=== Red-Black Tree ===

      56
     /  \
  12    61

Preorder Traversal:
56 12 61
InOrder Traversal:
12 56 61
Postorder Traversal:
12 61 56
```

4. Generate 10 Angka Acak (Opsi 2)

Program menghasilkan 10 angka acak (16 96 59 35 16 90 93 34 22 41), menyisipkan masing-masing angka tersebut ke dalam struktur. Secara fungsional fitur pembuatan data acak dan penambahan ke koleksi, berguna untuk pengujian operasi selanjutnya seperti traversal atau visualisasi pohon.

```
=== Red Black Tree Menu ===
1. Tambah data
2. Reset dan generate random
3. Show Tree & Traversals
4. Exit
==== Choose: 2
How many random numbers to generate? 10
Adding: 16 96 59 35 16 90 93 34 22 41
Tree regenerated.
==== Choose:
```

5. Visualisasi dan Traversals Angka Acak (Opsi 3)

Program mengosongkan pohon dengan `root = NIL` lalu melakukan *loop* sebanyak N (mis. 10) untuk mengambil nilai dari sumber (*random* atau *array*) dan memanggil `insert(value)`; setiap *insert* pertama-tama menempatkan node sesuai aturan *BST* (kiri jika lebih kecil, kanan jika lebih besar atau sesuai kebijakan duplikat), menginisialisasi node baru sebagai *RED*, menghubungkannya ke *parent*, lalu menjalankan `insertFixup` yang menangani tiga kasus utama (1) *uncle merah* → *recoloring parent* dan *uncle* menjadi *BLACK* dan *grandparent* menjadi *RED* lalu lanjut ke atas, (2) *uncle hitam* & node *inner* → rotasi pada *parent* untuk mengubah ke kasus

outer, dan (3) *uncle* hitam & node *outer* → rotasi pada *grandparent* dan tukar warna *parent/grandparent* setelah itu pastikan *root* berwarna *BLACK*; kebijakan duplikat (mis. selalu ke kanan) harus ditetapkan agar hasil deterministik; setelah semua nilai dimasukkan program menampilkan pesan konfirmasi *Tree regenerated*, mencetak visualisasi teks pohon (dengan indentasi dan warna tiap node) serta tiga traversal rekursif *Preorder*, *InOrder* (menghasilkan urutan menaik jika BST benar), dan *Postorder* untuk verifikasi struktur dan analisis urutan kunjungan.

```

==== Choose: 3

--- Visual Representation ---

=== Red-Black Tree ===

      34
     /  \
  16    59
 /  \  /  \
16 22 35 93
    /  \
  41  90 96

Preorder Traversal:
34 16 16 22 59 35 41 93 90 96
InOrder Traversal:
16 16 22 34 35 41 59 90 93 96
Postorder Traversal:
16 22 16 41 35 90 96 93 59 34
==== Choose:

```

6. Keluar (Opsi 4)

User memilih opsi 4 (Keluar) sehingga program berhenti dan menampilkan pesan sistem "*Process finished with exit code 0*", yang menandakan program telah berakhir dengan sukses tanpa *error*.

```

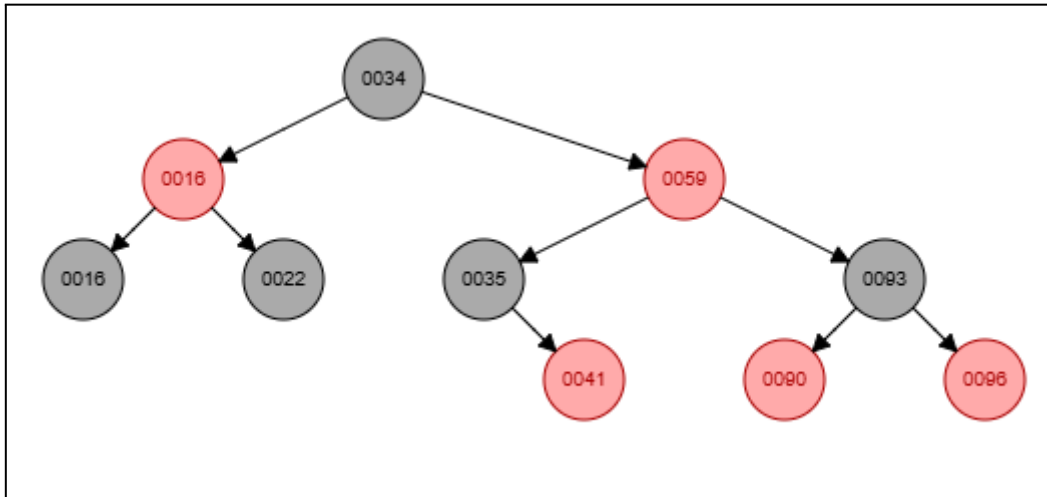
Preorder Traversal:
34 16 16 22 59 35 41 93 90 96
InOrder Traversal:
16 16 22 34 35 41 59 90 93 96
Postorder Traversal:
16 22 16 41 35 90 96 93 59 34
==== Choose: 4

Process finished with exit code 0

```

7. Visualisasi dari USFCA EDU

<https://www.cs.usfca.edu/~galles/visualization/RedBlack.html>



Refrensi

GeeksforGeeks. (2025, 3 November). Red Black Tree - GeeksforGeeks. Diakses 21 November 2025, dari <https://www.geeksforgeeks.org/dsa/introduction-to-red-black-tree/>

Galles, D. (n.d.). Red/Black Tree Visualization - University of San Francisco. Diakses 21 November 2025, dari <https://www.cs.usfca.edu/~galles/visualization/RedBlack.html>



Repository

<https://github.com/ZAZW27/RedBlackTree.git>